UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Ndox-SN

Intelligence Hydrique & Prédiction des Ruptures de Réseau

Eau & Assainissement

Durée	4 semaines
Équipe	2 étudiants
Domaine	Eau & Assainissement
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Ndox signifie « eau » en Wolof. Au Sénégal, 3 millions de personnes n'ont pas accès à l'eau potable, et les ruptures de réseau SONES/Sen Eau coûtent ~15 milliards FCFA/an. Ce projet construit un système de prédiction des ruptures imminentes du réseau de distribution, permettant une maintenance préventive et l'optimisation des camions citernes en zones rurales.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	capteurs pression SONES, données pluviométrie, consommation zones
2	**Prédiction des ruptures de réseau 48h à l'avance (classification par criticité)**	Prédiction des ruptures de réseau 48h à l'avance (classification par criticité)
3	**Optimisation des tournées de camions citernes en zones sans réseau**	Optimisation des tournées de camions citernes en zones sans réseau
4	**Privacy by Design**	anonymisation des compteurs résidentiels (données tarifaires)
5	**Dashboard SONES**	carte réseau, alertes rupture, recommandations maintenance

1.2 Architecture End-to-End

📄 Architecture

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 SONES capteurs Orchestration    Topic:           Prédiction         HBase/HDFS
 Pluviométrie   & Routage        ndox_pression    Rupture réseau     Gold Layer
 Consommation                    ndox_conso       Classification          |
                                                        |            [ ANALYTICS ]
                                                        ▼              Apache Hive
                                            [ ALERT LAYER ]        vue_reseau
                                            Rupture CRITIQUE       vue_citernes
```

2. Prérequis & Infrastructure

Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_ndox_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Ndox-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Ndox-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Ndox-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_ndox_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'ndox:capteurs_temps_reel': {'b'meta': {'max_versions': 1}},  # Pression réseau
par zone (TTL 24h)
        b'ndox:alertes_rupture': {'b'meta': {'max_versions': 1}},  # Alertes criticité
(TTL 48h)
        b'ndox:tournees_citernes': {'b'meta': {'max_versions': 1}},  # Optimisation
tournées zones rurales
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_ndox_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Eau & Assainissement au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_ndox_sn.py**

```
# kafka_producer_ndox.py — Simulateur capteurs réseau eau + consommation
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
ZONES = ['DAKAR_NORD','DAKAR_SUD','PIKINE','THIES','SAINT_LOUIS','KAOLACK','ZIGUINCHOR']
# Pression nominale par zone (bars) et fragilité du réseau
PRESSION_NOMINALE = {'DAKAR_NORD':4.5,'DAKAR_SUD':4.2,'PIKINE':3.8,
                     'THIES':3.5,'SAINT_LOUIS':3.2,'KAOLACK':3.0,'ZIGUINCHOR':2.8}
FRAGILITE_RESEAU  = {'DAKAR_NORD':0.05,'DAKAR_SUD':0.08,'PIKINE':0.20,
                     'THIES':0.15,'SAINT_LOUIS':0.18,'KAOLACK':0.22,'ZIGUINCHOR':0.25}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode())
```

```
def gen_pression(zone):
    p_nom  = PRESSION_NOMINALE[zone]
    frag   = FRAGILITE_RESEAU[zone]
    # Simulation chute pression anormale (événement rare)
    if random.random() < frag:
        # Rupture imminente
        pression = round(max(0.1, p_nom * random.uniform(0.2, 0.5)), 2)
        statut   = 'RUPTURE_IMMINENTE'
    elif random.random() < 0.1:
        pression = round(p_nom * random.uniform(0.6, 0.8), 2)
        statut   = 'PRESSION_BASSE'
    else:
        pression = round(np.random.normal(p_nom, p_nom*0.05), 2)
        statut   = 'NORMAL'
    return {
        'capteur_id':          f'CPT_{zone}_{uuid.uuid4().hex[:8]}',  # Sera anonymisé
        'zone':                zone,
        'pression_bars':       pression,
        'pression_nominale':   p_nom,
        'debit_m3h':           round(random.uniform(50, 500) * (pression/p_nom), 1),
        'statut_capteur':      statut,
        'age_canalisation_ans':round(random.uniform(5, 45), 1),
        'timestamp':           datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

def gen_conso(zone):
    h = datetime.utcnow().hour
    # Pic consommation matin 6-8h et soir 18-20h
    facteur = 1.5 if (6<=h<=8 or 18<=h<=20) else 0.8
    return {
        'compteur_id':  f'CMT_{uuid.uuid4().hex[:10]}',  # Sera supprimé
        'adresse_client': f'Rue {random.randint(1,500)} {zone}',  # Sera supprimée
        'zone':         zone,
        'consommation_m3': round(random.uniform(0.5, 5.0) * facteur, 2),
        'heure':        h,
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Ndox-SN démarré...')
    while True:
        for zone in ZONES:
            producer.send('ndox_pression_raw', gen_pression(zone))
            producer.send('ndox_conso_raw',    gen_conso(zone))
        producer.flush(); time.sleep(5)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_ndox_sn.py

```
# streaming_ndox.py — Pipeline Spark Streaming Ndox-SN
# Détection chute pression anormale + prédiction rupture
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, stddev, min as spark_min, when, coalesce, current_timestamp
)
from pyspark.sql.types import *

SALT = os.environ.get('NDOX_SECRET_SALT','default_salt')
spark = SparkSession.builder.appName('Ndox_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

press_schema = StructType([
    StructField('capteur_id',           StringType(), True),
    StructField('zone',                 StringType(), True),
    StructField('pression_bars',        FloatType(),  True),
    StructField('pression_nominale',    FloatType(),  True),
    StructField('debit_m3h',            FloatType(),  True),
    StructField('statut_capteur',       StringType(), True),
    StructField('age_canalisation_ans', FloatType(),  True),
    StructField('timestamp',            StringType(), True),
])

press_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','ndox_pression_raw').load()
    .select(from_json(col('value').cast('string'), press_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    .withColumn('capteur_secure', sha2(concat(col('capteur_id'),lit(SALT)),256))
    .drop('capteur_id')
)

# Détection anomalie pression (fenêtre 1h, watermark 10min)
anomalie_df = (press_df
    .withWatermark('event_ts','10 minutes')
    .groupBy(window('event_ts','1 hour','15 minutes'), 'zone')
    .agg(
        avg('pression_bars').alias('pression_moy'),
        spark_min('pression_bars').alias('pression_min'),
        avg('pression_nominale').alias('pression_nominale_moy'),
        avg('debit_m3h').alias('debit_moy'),
    )
    .withColumn('ratio_pression',
        coalesce(col('pression_moy'),lit(3.0)) /
        coalesce(col('pression_nominale_moy'),lit(4.0)))
    .withColumn('criticite_rupture',
        when(col('ratio_pression') < 0.3, lit('CRITIQUE'))
        .when(col('ratio_pression') < 0.6, lit('ALERTE'))
        .when(col('ratio_pression') < 0.8, lit('SURVEILLANCE'))
        .otherwise(lit('NORMAL')))
)

q1 = (anomalie_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','ndox_alerts')
    .option('checkpointLocation','/tmp/ndox_press_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Ndox-SN
CREATE DATABASE IF NOT EXISTS ndox_sn COMMENT 'Eau Sénégal — UADB 2025';
USE ndox_sn;

CREATE TABLE IF NOT EXISTS pression_historique (
    capteur_secure STRING, zone STRING,
    pression_bars DOUBLE, pression_nominale DOUBLE,
    debit_m3h DOUBLE, statut_capteur STRING,
    age_canalisation_ans DOUBLE, ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS interventions_maintenance (
    zone STRING, type_intervention STRING,
    date_intervention STRING, cout_fcfa BIGINT,
    rupture_evitee BOOLEAN
) STORED AS ORC;

-- Vue : État du réseau par zone
CREATE OR REPLACE VIEW vue_etat_reseau AS
SELECT zone,
    AVG(pression_bars / COALESCE(pression_nominale,4.0)) AS ratio_pression_moy,
    MIN(pression_bars)                                   AS pression_mini,
    AVG(age_canalisation_ans)                            AS age_moy_canalisation,
    SUM(CASE WHEN statut_capteur='RUPTURE_IMMINENTE' THEN 1 ELSE 0 END) AS nb_alertes_24h,
    CASE
        WHEN AVG(pression_bars/COALESCE(pression_nominale,4.0)) < 0.4 THEN 'CRITIQUE'
        WHEN AVG(pression_bars/COALESCE(pression_nominale,4.0)) < 0.7 THEN 'ALERTE'
        ELSE 'NORMAL'
    END AS statut_zone
FROM pression_historique
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),1)
GROUP BY zone ORDER BY ratio_pression_moy ASC;

-- Vue : Zones prioritaires pour maintenance
CREATE OR REPLACE VIEW vue_maintenance_prioritaire AS
SELECT zone, AVG(age_canalisation_ans) AS age_moy,
    COUNT(CASE WHEN statut_capteur != 'NORMAL' THEN 1 END) AS nb_incidents,
    AVG(age_canalisation_ans) * 0.4 +
    COUNT(CASE WHEN statut_capteur != 'NORMAL' THEN 1 END) * 0.6 AS score_priorite
FROM pression_historique
GROUP BY zone
ORDER BY score_priorite DESC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/ndox_sn_dag.py

```
# dags/ndox_sn_dag.py — DAG Airflow Ndox-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging
```

```
logger = logging.getLogger('ndox_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}

def check_ruptures(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore',port=10000)
    cur  = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM ndox_sn.vue_etat_reseau WHERE
statut_zone='CRITIQUE'")
    n = cur.fetchone()[0]
    return 'retrain_model' if n > 1 else 'update_alerts'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_rupture_model.py'],check=True,timeout=3600)

def update_alerts(**ctx):
    import happybase; from pyhive import hive; from datetime import datetime
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT zone,ratio_pression_moy,nb_alertes_24h,statut_zone FROM
ndox_sn.vue_etat_reseau')
    table = conn_hbase.table(b'ndox:alertes_rupture')
    for row in cur.fetchall():
        zone,ratio,nb,statut = row
        if statut != 'NORMAL':
            table.put(zone.encode(),{
                b'alerte:zone':zone.encode(),
b'alerte:ratio':str(round(ratio,3)).encode(),
                b'alerte:statut':statut.encode(),
b'alerte:ts':datetime.utcnow().isoformat().encode()})
    conn_hbase.close()

with DAG('ndox_sn_monitoring', default_args=default_args,
         schedule_interval='0 */4 * * *', start_date=days_ago(1), catchup=False,
         tags=['ndox-sn','eau','assainissement']) as dag:
    start  = DummyOperator(task_id='start'); end = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_ruptures',
                                  python_callable=check_ruptures,provide_context=True)
    t_train=
PythonOperator(task_id='retrain_model',python_callable=retrain_model,provide_context=True)
    t_alert=
PythonOperator(task_id='update_alerts',python_callable=update_alerts,provide_context=True)
    start >> branch >> [t_train, t_alert]
    t_train >> t_alert >> end; t_alert >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka ndox_pression_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Eau & Assainissement sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	**3 pts**	NiFi absent	1 source/3	SONES + pluviométrie	3 sources, 2 topics, messages avec pression et coordonnées GPS
Validation & Privacy (Compteurs)	**4 pts**	Aucune validation	Schéma sans anonymisation	SHA-256 sans drop compteur_id	Pandera + drop(compteur_id, adresse_client) + 0 PII dans HBase
Streaming & Détection Rupture	**6 pts**	Pas de Spark	Lecture seule	Seuil pression fixe	Chute pression anormale (AVG-3*STDDEV) + watermarks + COALESCE
Prédiction 48h & MLOps	**4 pts**	Aucun modèle	Règles expertes	RF entraîné sans DAG	Gradient Boosting + DAG quotidien + recall ruptures documenté
Tournées Citernes & Dashboard	**3 pts**	Pas de dashboard	KPIs réseau vides	Carte ruptures sans optimisation	Optimisation tournées citernes (distance minimale) + alertes SONES live
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

💧 **Bon courage pour Ndox-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel